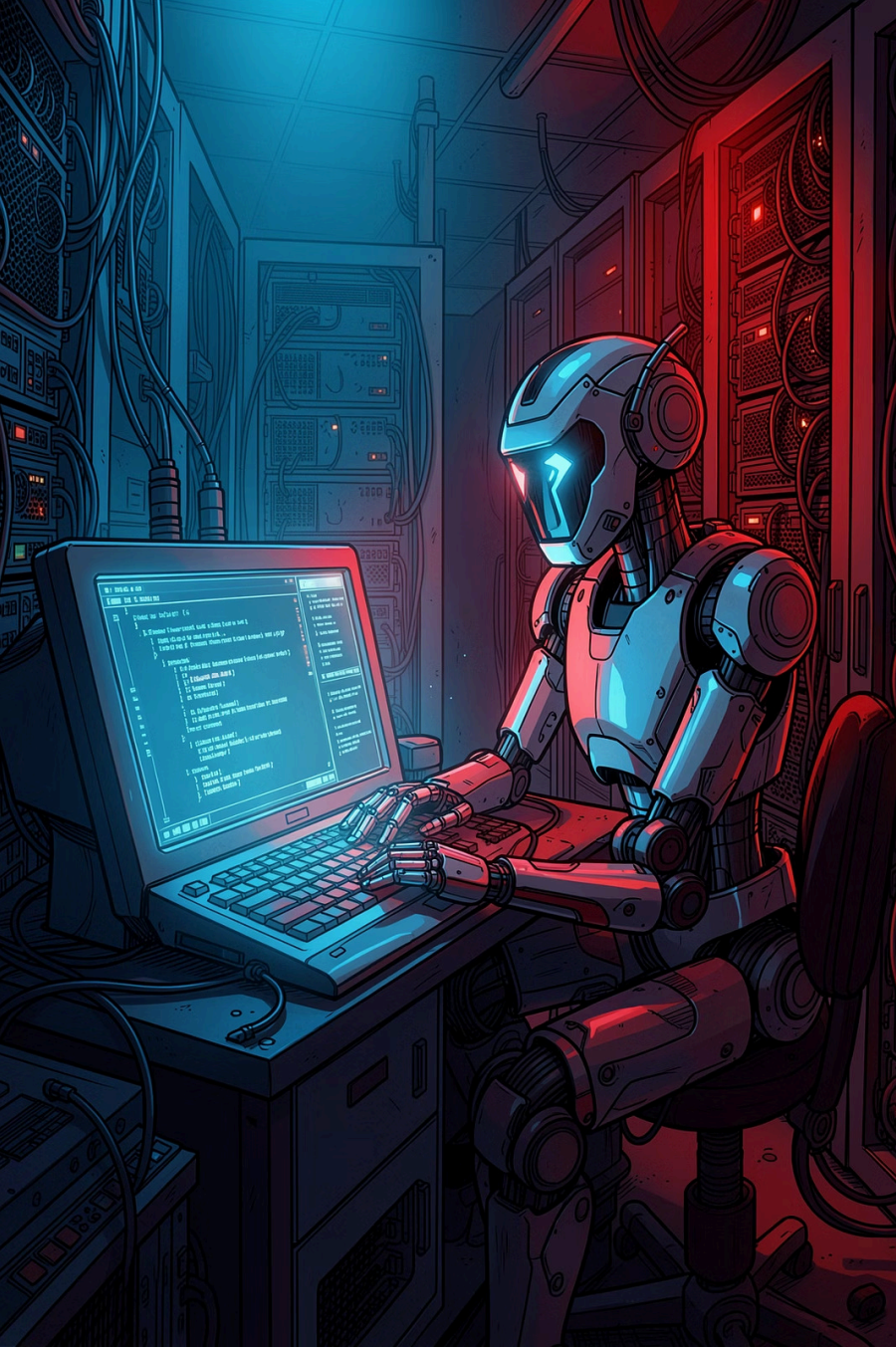




The Code Execution Hazards of Autonomous Agents

When AI agents write and execute their own code, the risks are not just logical — they are systemic. Three failure modes threaten stability, observability, and progress in every autonomous development pipeline.

Three Critical Failure Modes

Runaway Code Loop

Accidental infinite loops and deep recursion consume 100% CPU, freezing the execution environment entirely.

Silent OOM Kills

Uncontrolled memory allocation triggers OS-level termination with zero diagnostic logs — the agent cannot learn from failure.

Goal Drift & Iteration Stalls

Without boundary monitors, agents trap themselves in endless bug-fixing cycles, regenerating identical invalid code.

The Runaway Code Loop



AI models frequently generate constructs like `while True: pass` or unbounded recursion. These patterns consume **100% of available CPU capacity**, locking the execution environment and halting all development tasks.

⚠ Unlike human-written bugs, these loops are generated confidently — the model has no intrinsic signal that execution has stalled.

Why Infinite Loops Are So Dangerous

No Self-Termination Signal

The agent receives no error — the process is "running successfully" from its perspective, even as it consumes all resources.

Environment-Wide Impact

A single runaway loop can freeze shared sandboxes, blocking parallel tasks and disrupting entire CI/CD pipelines.

Hard to Detect Automatically

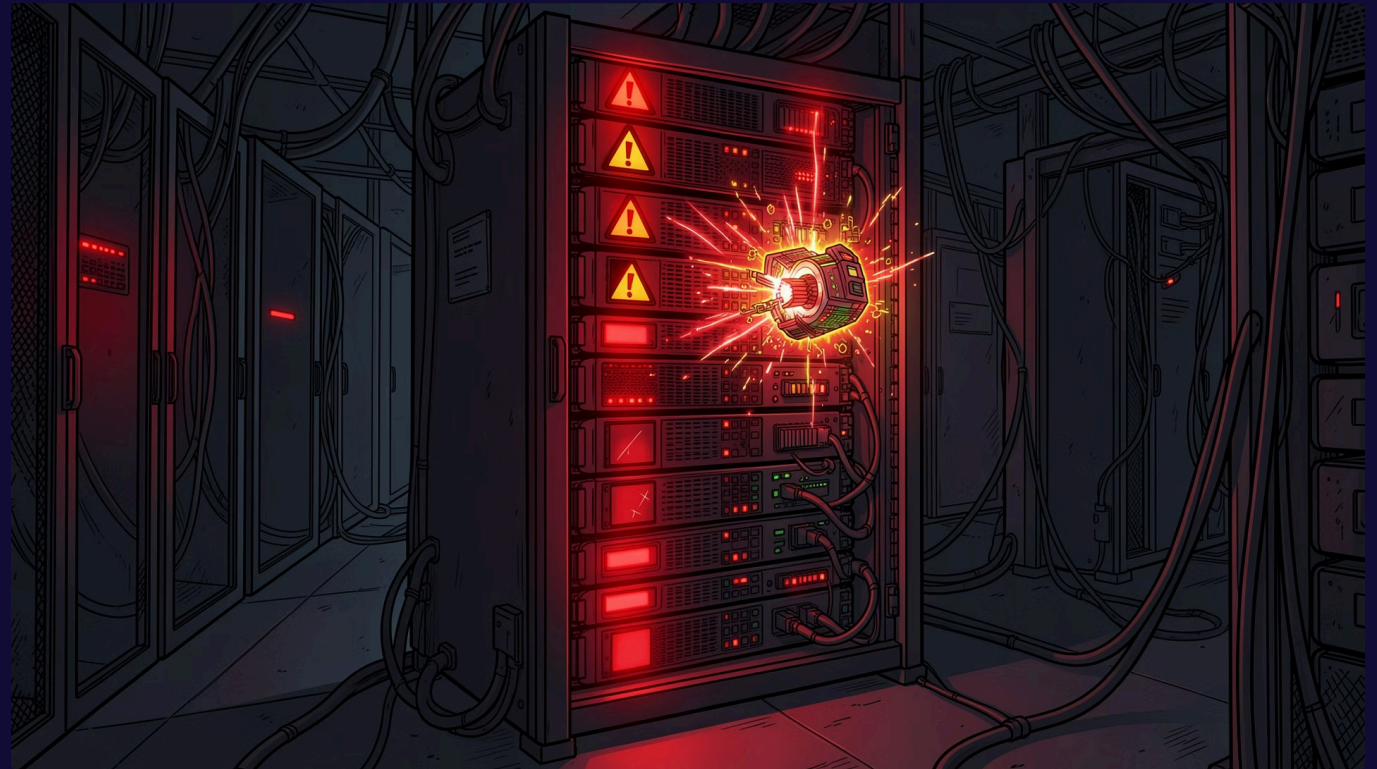
Static analysis often misses dynamically generated loop conditions. Runtime watchdogs are essential but rarely configured by default.

Silent Out-of-Memory Kills

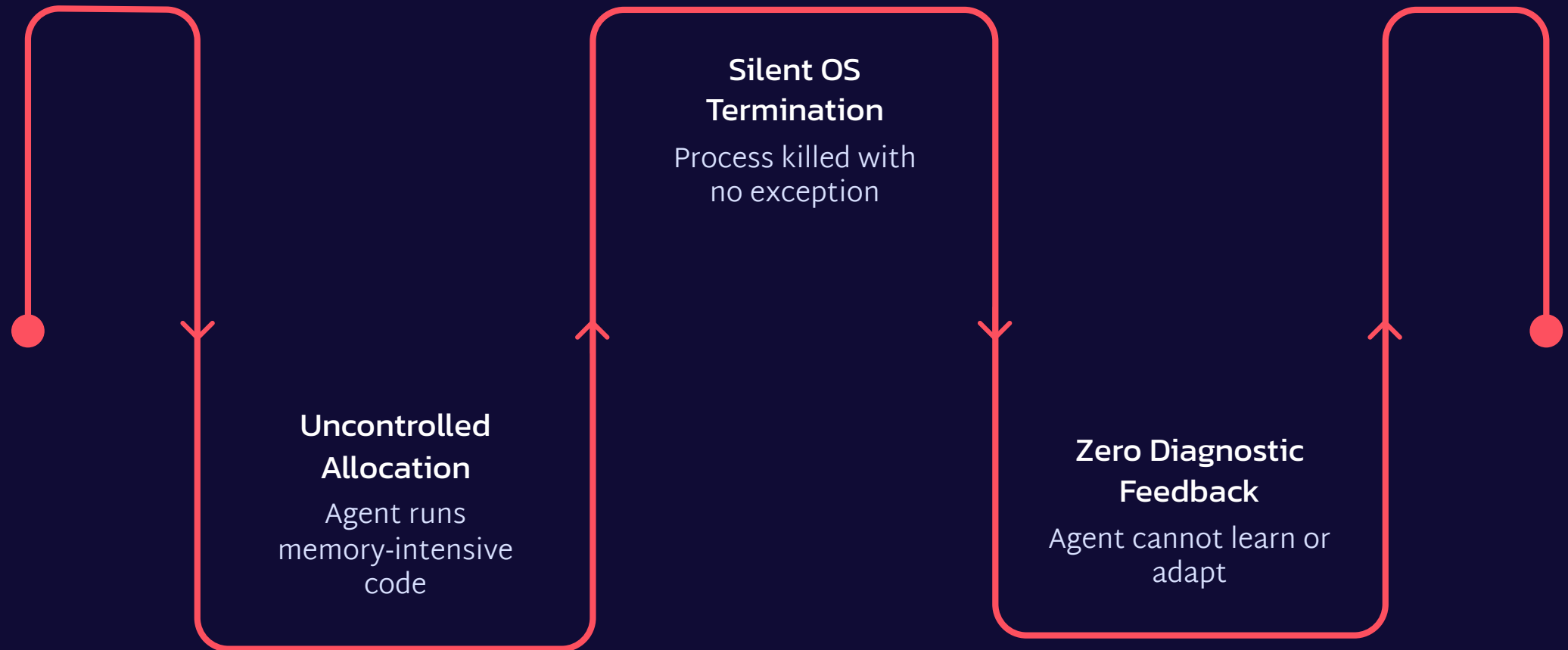
The Invisible Failure

Heavy mathematical operations and large data processing tasks can allocate memory uncontrollably. When thresholds are breached, the **OS silently terminates the process** — no exception, no stack trace, no log entry.

The agent is left blind, unable to distinguish a successful run from a killed one, and cannot adapt its behavior.



Why OOM Kills Are a Learning Black Hole

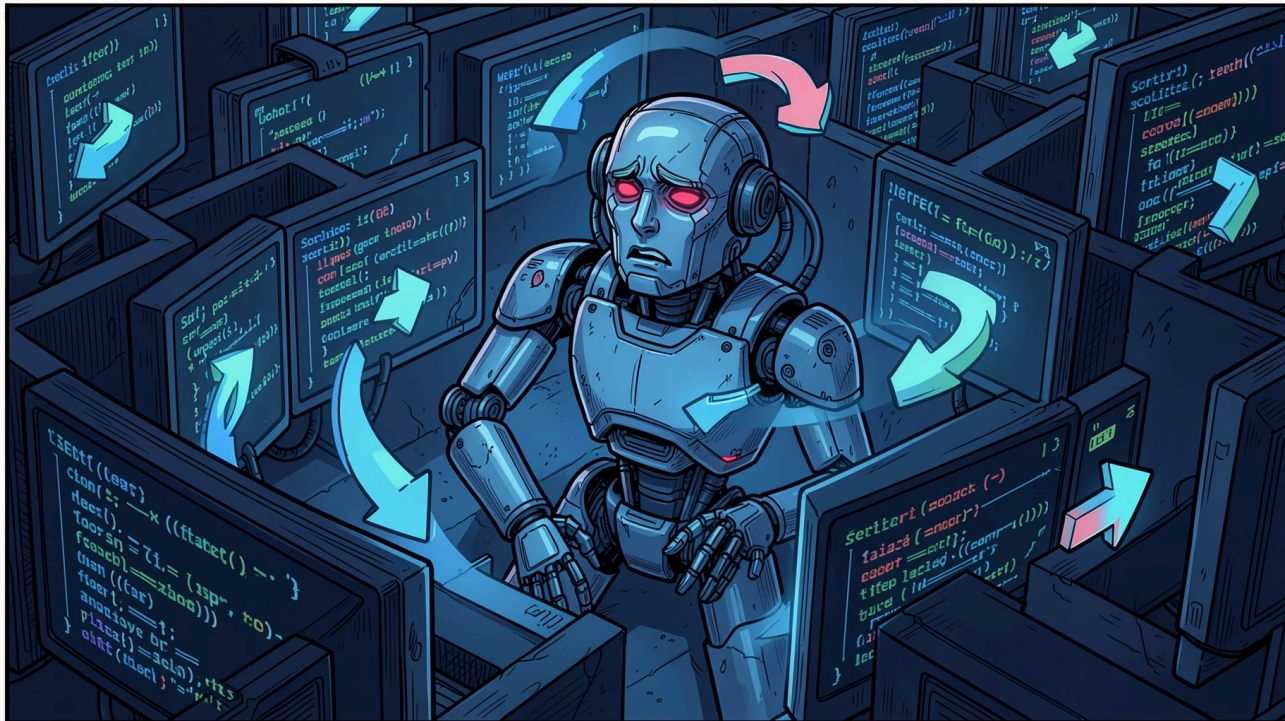


Without observable failure signals, the agent cannot update its generation strategy. The same memory-intensive pattern may be regenerated indefinitely — a silent, unrecoverable loop at the system level.

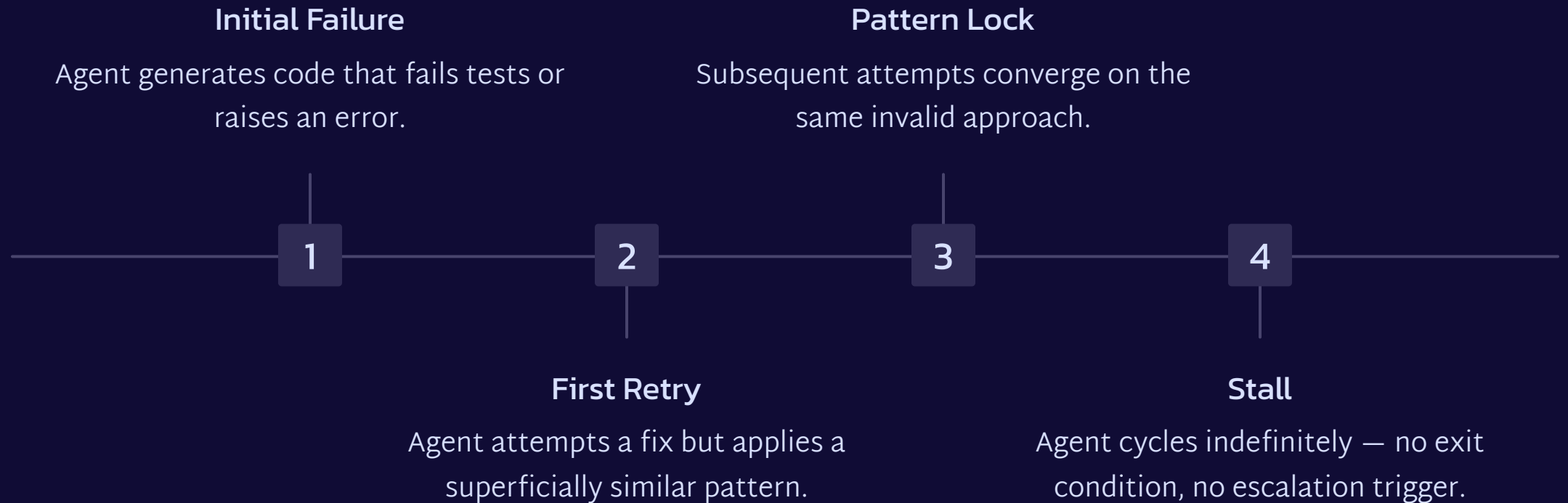
Goal Drift & Infinite Iteration Stalls

Without strict logic boundary monitors, developer agents become trapped in **endless bug-fixing loops** — generating the same invalid code patterns repeatedly without detecting failure.

- The agent "believes" it is making progress on each iteration
- No convergence detection or iteration counter is enforced
- Task context degrades as the loop extends
- Human intervention becomes the only escape hatch



The Anatomy of a Goal Drift Stall



- ① Goal drift is a reasoning failure, not just an execution failure. The agent needs meta-cognitive boundaries, not just runtime guards.

Building Guardrails: What Engineers Must Do



Enforce Execution Timeouts

Hard CPU-time limits on every code execution block, with automatic termination and structured error capture.



Monitor Memory Budgets

Set per-process memory ceilings and surface OOM events as observable signals the agent can act upon.



Implement Iteration Limits

Track retry counts per task, detect pattern repetition, and trigger escalation or human handoff thresholds.



Key Takeaways

Runaway Loops

Enforce hard execution timeouts and watchdog timers on every agent-generated code block.

Silent OOM Kills

Make memory limits observable — surface termination events as structured feedback the agent can learn from.

Goal Drift

Implement iteration counters, convergence detection, and escalation triggers to break infinite retry cycles.

❏ **Bottom line:** Autonomous agents need runtime guardrails as much as they need capable models. Safety is an infrastructure problem, not just a prompting problem.